8.1 General Trees

Productivity experts say that breakthroughs come by thinking "nonlinearly." In this chapter, we discuss one of the most important nonlinear data structures in computing—***trees***. Tree structures are indeed a breakthrough in data organization, for they allow us to implement a host of algorithms much faster than when using linear data structures, such as array-based lists or linked lists. Trees also provide a natural organization for data, and consequently have become ubiquitous structures in file systems, graphical user interfaces, databases, Web sites, and other computer systems.

It is not always clear what productivity experts mean by "nonlinear" thinking, but when we say that trees are "nonlinear," we are referring to an organizational relationship that is richer than the simple "before" and "after" relationships between objects in sequences. The relationships in a tree are ***hierarchical***, with some objects being "above" and some "below" others. Actually, the main terminology for tree data structures comes from family trees, with the terms "parent," "child," "ancestor," and "descendant" being the most common words used to describe relationships. We show an example of a family tree in Figure 8.1.

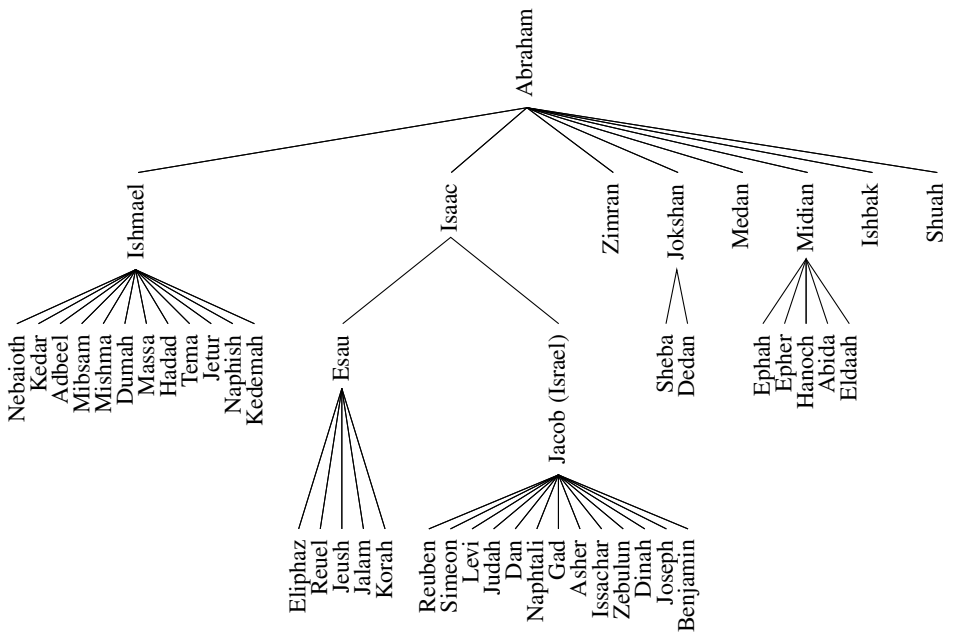


Figure 8.1: A family tree showing some descendants of Abraham, as recorded in Genesis, chapters 25–36.

8.1.1 Tree Definitions and Properties

A ***tree*** is an abstract data type that stores elements hierarchically. With the exception of the top element, each element in a tree has a ***parent*** element and zero or more ***children*** elements. A tree is usually visualized by placing elements inside ovals or rectangles, and by drawing the connections between parents and children with straight lines. (See Figure 8.2.) We typically call the top element the ***root*** of the tree, but it is drawn as the highest element, with the other elements being connected below (just the opposite of a botanical tree).

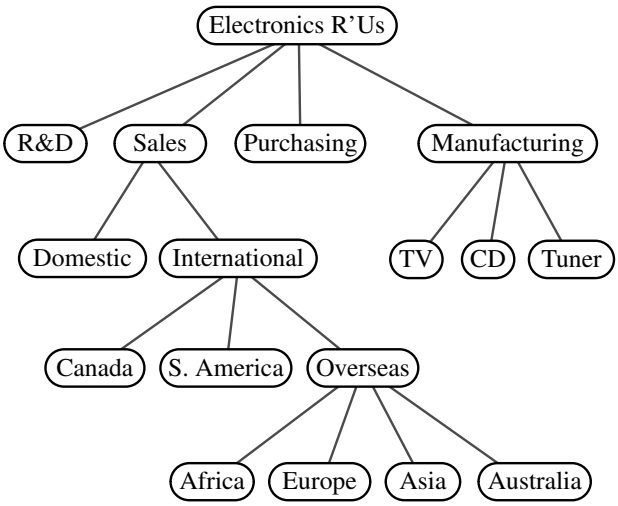


Figure 8.2: A tree with 17 nodes representing the organization of a fictitious corporation. The root stores *Electronics R'Us*. The children of the root store *R&D*, *Sales*, *Purchasing*, and *Manufacturing*. The internal nodes store *Sales*, *International*, *Overseas*, *Electronics R'Us*, and *Manufacturing*.

Formal Tree Definition

Formally, we define a ***tree*** T as a set of ***nodes*** storing elements such that the nodes have a ***parent-child*** relationship that satisfies the following properties:

- If T is nonempty, it has a special node, called the ***root*** of T, that has no parent.
- Each node v of T different from the root has a unique ***parent*** node w; every node with parent w is a ***child*** of w.

Note that according to our definition, a tree can be empty, meaning that it does not have any nodes. This convention also allows us to define a tree recursively such that a tree T is either empty or consists of a node r, called the root of T, and a (possibly empty) set of subtrees whose roots are the children of r.

Other Node Relationships

Two nodes that are children of the same parent are ***siblings***. A node v is ***external*** if v has no children. A node v is ***internal*** if it has one or more children. External nodes are also known as ***leaves***.

Example 8.1: *In Section 4.1.4, we discussed the hierarchical relationship between files and directories in a computer's file system, although at the time we did not emphasize the nomenclature of a file system as a tree. In Figure 8.3, we revisit an earlier example. We see that the internal nodes of the tree are associated with directories and the leaves are associated with regular files. In the UNIX and Linux operating systems, the root of the tree is appropriately called the "root directory," and is represented by the symbol "/."*

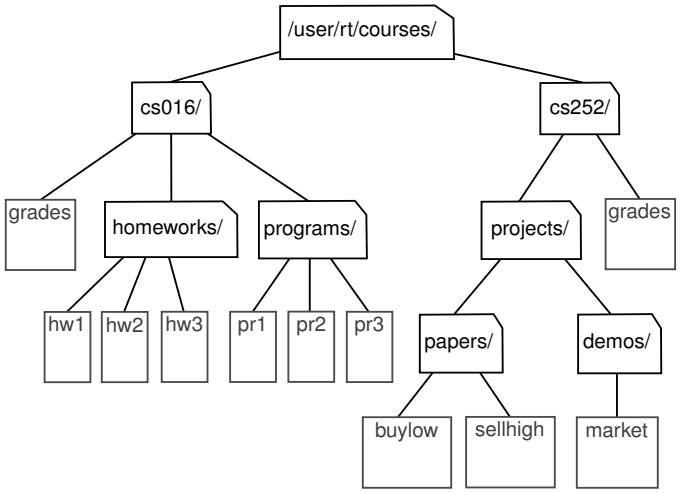


Figure 8.3: Tree representing a portion of a file system.

A node u is an ***ancestor*** of a node v if $u = v$ or u is an ancestor of the parent of v. Conversely, we say that a node v is a ***descendant*** of a node u if u is an ancestor of v. For example, in Figure 8.3, cs252/ is an ancestor of papers/, and pr3 is a descendant of cs016/. The ***subtree*** of T ***rooted*** at a node v is the tree consisting of all the descendants of v in T (including v itself). In Figure 8.3, the subtree rooted at cs016/ consists of the nodes cs016/, grades, homeworks/, programs/, hw1, hw2, hw3, pr1, pr2, and pr3.

Edges and Paths in Trees

An ***edge*** of tree T is a pair of nodes (u,v) such that u is the parent of v, or vice versa. A ***path*** of T is a sequence of nodes such that any two consecutive nodes in the sequence form an edge. For example, the tree in Figure 8.3 contains the path (cs252/, projects/, demos/, market).

Example 8.2: *The inheritance relation between classes in a Python program forms a tree when single inheritance is used. For example, in Section 2.4 we provided a summary of the hierarchy for Python's exception types, as portrayed in Figure 8.4 (originally Figure 2.5). The* BaseException *class is the root of that hierarchy, while all user-defined exception classes should conventionally be declared as descendants of the more specific* Exception *class. (See, for example, the* Empty *class we introduced in Code Fragment 6.1 of Chapter 6.)*

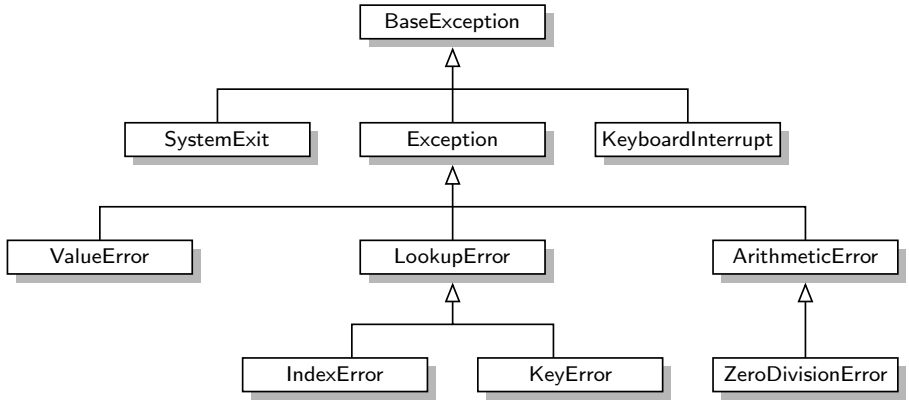


Figure 8.4: A portion of Python's hierarchy of exception types.

In Python, all classes are organized into a single hierarchy, as there exists a built-in class named object as the ultimate base class. It is a direct or indirect base class of all other types in Python (even if not declared as such when defining a new class). Therefore, the hierarchy pictured in Figure 8.4 is only a portion of Python's complete class hierarchy.

As a preview of the remainder of this chapter, Figure 8.5 portrays our own hierarchy of classes for representing various forms of a tree.

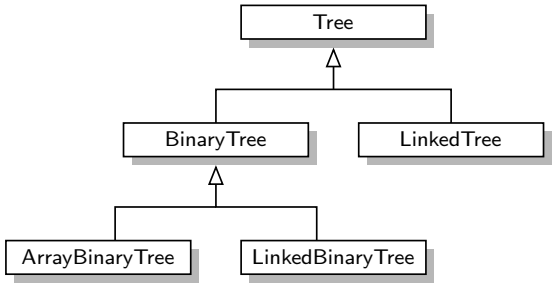


Figure 8.5: Our own inheritance hierarchy for modeling various abstractions and implementations of tree data structures. In the remainder of this chapter, we provide implementations of Tree, BinaryTree, and LinkedBinaryTree classes, and high-level sketches for how LinkedTree and ArrayBinaryTree might be designed.

Ordered Trees

A tree is ***ordered*** if there is a meaningful linear order among the children of each node; that is, we purposefully identify the children of a node as being the first, second, third, and so on. Such an order is usually visualized by arranging siblings left to right, according to their order.

Example 8.3: *The components of a structured document, such as a book, are hierarchically organized as a tree whose internal nodes are parts, chapters, and sections, and whose leaves are paragraphs, tables, figures, and so on. (See Figure 8.6.) The root of the tree corresponds to the book itself. We could, in fact, consider expanding the tree further to show paragraphs consisting of sentences, sentences consisting of words, and words consisting of characters. Such a tree is an example of an ordered tree, because there is a well-defined order among the children of each node.*

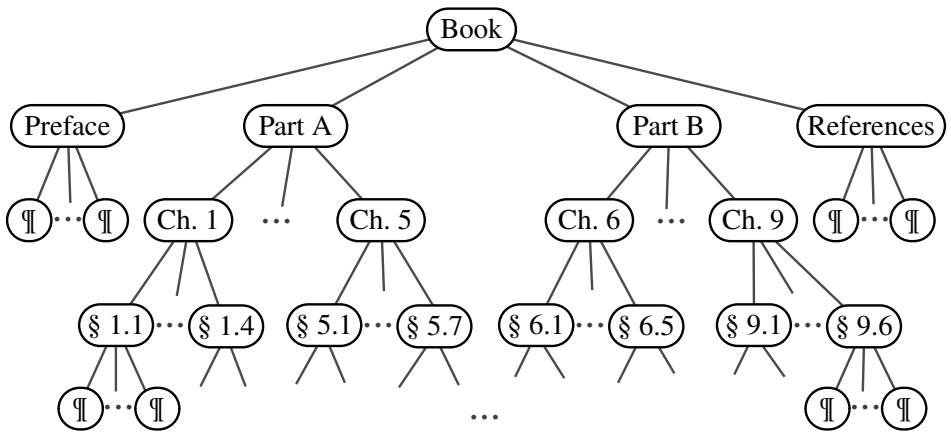


Figure 8.6: An ordered tree associated with a book.

Let's look back at the other examples of trees that we have described thus far, and consider whether the order of children is significant. A family tree that describes generational relationships, as in Figure 8.1, is often modeled as an ordered tree, with siblings ordered according to their birth.

In contrast, an organizational chart for a company, as in Figure 8.2, is typically considered an unordered tree. Likewise, when using a tree to describe an inheritance hierarchy, as in Figure 8.4, there is no particular significance to the order among the subclasses of a parent class. Finally, we consider the use of a tree in modeling a computer's file system, as in Figure 8.3. Although an operating system often displays entries of a directory in a particular order (e.g., alphabetical, chronological), such an order is not typically inherent to the file system's representation.

8.1.2 The Tree Abstract Data Type

As we did with positional lists in Section 7.4, we define a tree ADT using the concept of a ***position*** as an abstraction for a node of a tree. An element is stored at each position, and positions satisfy parent-child relationships that define the tree structure. A position object for a tree supports the method:

p.element( ): Return the element stored at position p.

The tree ADT then supports the following ***accessor methods***, allowing a user to navigate the various positions of a tree:

T.root( ): Return the position of the root of tree T, or None if T is empty.

T.is_root(p): Return True if position p is the root of Tree T.

T.parent(p): Return the position of the parent of position p, or None if p is the root of T.

T.num_children(p): Return the number of children of position p.

T.children(p): Generate an iteration of the children of position p.

T.is_leaf(p): Return True if position p does not have any children.

len(T): Return the number of positions (and hence elements) that are contained in tree T.

T.is_empty( ): Return True if tree T does not contain any positions.

T.positions( ): Generate an iteration of all *positions* of tree T.

iter(T): Generate an iteration of all *elements* stored within tree T.

Any of the above methods that accepts a position as an argument should generate a ValueError if that position is invalid for T.

If a tree T is ordered, then T.children(p) reports the children of p in the natural order. If p is a leaf, then T.children(p) generates an empty iteration. In similar regard, if tree T is empty, then both T.positions( ) and iter(T) generate empty iterations. We will discuss general means for iterating through all positions of a tree in Sections 8.4.

We do not define any methods for creating or modifying trees at this point. We prefer to describe different tree update methods in conjunction with specific implementations of the tree interface, and specific applications of trees.

A Tree Abstract Base Class in Python

In discussing the object-oriented design principle of abstraction in Section 2.1.2, we noted that a public interface for an abstract data type is often managed in Python via ***duck typing***. For example, we defined the notion of the public interface for a queue ADT in Section 6.2, and have since presented several classes that implement the queue interface (e.g., ArrayQueue in Section 6.2.2, LinkedQueue in Section 7.1.2, CircularQueue in Section 7.2.2). However, we never gave any formal definition of the queue ADT in Python; all of the concrete implementations were self-contained classes that just happen to adhere to the same public interface. A more formal mechanism to designate the relationships between different implementations of the same abstraction is through the definition of one class that serves as an ***abstract base class***, via inheritance, for one or more ***concrete classes***. (See Section 2.4.3.)

We choose to define a Tree class, in Code Fragment 8.1, that serves as an abstract base class corresponding to the tree ADT. Our reason for doing so is that there is quite a bit of useful code that we can provide, even at this level of abstraction, allowing greater code reuse in the concrete tree implementations we later define. The Tree class provides a definition of a nested Position class (which is also abstract), and declarations of many of the accessor methods included in the tree ADT.

However, our Tree class does not define any internal representation for storing a tree, and five of the methods given in that code fragment remain ***abstract*** (root, parent, num_children, children, and __len__); each of these methods raises a NotImplementedError. (A more formal approach for defining abstract base classes and abstract methods, using Python's abc module, is described in Section 2.4.3.) The subclasses are responsible for overriding abstract methods, such as children, to provide a working implementation for each behavior, based on their chosen internal representation.

Although the Tree class is an abstract base class, it includes several ***concrete*** methods with implementations that rely on calls to the abstract methods of the class. In defining the tree ADT in the previous section, we declare ten accessor methods. Five of those are the ones we left as abstract, in Code Fragment 8.1. The other five can be implemented based on the former. Code Fragment 8.2 provides concrete implementations for methods is_root, is_leaf, and is_empty. In Section 8.4, we will explore general algorithms for traversing a tree that can be used to provide concrete implementations of the positions and __iter__ methods within the Tree class. The beauty of this design is that the concrete methods defined within the Tree abstract base class will be inherited by all subclasses. This promotes greater code reuse, as there will be no need for those subclasses to reimplement such behaviors.

We note that, with the Tree class being abstract, there is no reason to create a direct instance of it, nor would such an instance be useful. The class exists to serve as a base for inheritance, and users will create instances of concrete subclasses.

```
class Tree:
  """Abstract base class representing a tree structure."""

  #-------------------------------- nested Position class --------------------------------
  class Position:
    """An abstraction representing the location of a single element."""

    def element(self):
      """Return the element stored at this Position."""
      raise NotImplementedError('must be implemented by subclass')

    def __eq__(self, other):
      """Return True if other Position represents the same location."""
      raise NotImplementedError('must be implemented by subclass')

    def __ne__(self, other):
      """Return True if other does not represent the same location."""
      return not (self == other)                # opposite of __eq__

  # ---------- abstract methods that concrete subclass must support ----------
  def root(self):
    """Return Position representing the tree's root (or None if empty)."""
    raise NotImplementedError('must be implemented by subclass')

  def parent(self, p):
    """Return Position representing p's parent (or None if p is root)."""
    raise NotImplementedError('must be implemented by subclass')

  def num_children(self, p):
    """Return the number of children that Position p has."""
    raise NotImplementedError('must be implemented by subclass')

  def children(self, p):
    """Generate an iteration of Positions representing p's children."""
    raise NotImplementedError('must be implemented by subclass')

  def __len__(self):
    """Return the total number of elements in the tree."""
    raise NotImplementedError('must be implemented by subclass')
```

Code Fragment 8.1: A portion of our Tree abstract base class (continued in Code Fragment 8.2).

```
40    # ---------- concrete methods implemented in this class ----------
41    def is_root(self, p):
42      """Return True if Position p represents the root of the tree."""
43      return self.root( ) == p
44
45    def is_leaf(self, p):
46      """Return True if Position p does not have any children."""
47      return self.num_children(p) == 0
48
49    def is_empty(self):
50      """Return True if the tree is empty."""
51      return len(self) == 0
```

Code Fragment 8.2: Some concrete methods of our Tree abstract base class.

8.1.3 Computing Depth and Height

Let p be the position of a node of a tree T. The ***depth*** of p is the number of ancestors of p, excluding p itself. For example, in the tree of Figure 8.2, the node storing *International* has depth 2. Note that this definition implies that the depth of the root of T is 0. The depth of p can also be recursively defined as follows:

- If p is the root, then the depth of p is 0.
- Otherwise, the depth of p is one plus the depth of the parent of p.

Based on this definition, we present a simple, recursive algorithm, depth, in Code Fragment 8.3, for computing the depth of a position p in Tree T. This method calls itself recursively on the parent of p, and adds 1 to the value returned.

```
52    def depth(self, p):
53      """Return the number of levels separating Position p from the root."""
54      if self.is_root(p):
55        return 0
56      else:
57        return 1 + self.depth(self.parent(p))
```

Code Fragment 8.3: Method depth of the Tree class.

The running time of T.depth(p) for position p is $O(d_p + 1)$, where d_p denotes the depth of p in the tree T, because the algorithm performs a constant-time recursive step for each ancestor of p. Thus, algorithm T.depth(p) runs in $O(n)$ worst-case time, where n is the total number of positions of T, because a position of T may have depth $n-1$ if all nodes form a single branch. Although such a running time is a function of the input size, it is more informative to characterize the running time in terms of the parameter d_p, as this parameter may be much smaller than n.

Height

The ***height*** of a position p in a tree T is also defined recursively:

- If p is a leaf, then the height of p is 0.
- Otherwise, the height of p is one more than the maximum of the heights of p's children.

The ***height*** of a nonempty tree T is the height of the root of T. For example, the tree of Figure 8.2 has height 4. In addition, height can also be viewed as follows.

Proposition 8.4: *The height of a nonempty tree T is equal to the maximum of the depths of its leaf positions.*

We leave the justification of this fact to an exercise (R-8.3). We present an algorithm, height1, implemented in Code Fragment 8.4 as a nonpublic method _height1 of the Tree class. It computes the height of a nonempty tree T based on Proposition 8.4 and the algorithm depth from Code Fragment 8.3.

```
58   def _height1(self):                    # works, but O(n^2) worst-case time
59     """Return the height of the tree."""
60     return max(self.depth(p) for p in self.positions( ) if self.is_leaf(p))
```

Code Fragment 8.4: Method _height1 of the Tree class. Note that this method calls the depth method.

Unfortunately, algorithm height1 is not very efficient. We have not yet defined the positions( ) method; we will see that it can be implemented to run in $O(n)$ time, where n is the number of positions of T. Because height1 calls algorithm depth(p) on each leaf of T, its running time is $O(n + \sum_{p \in L}(d_p + 1))$, where L is the set of leaf positions of T. In the worst case, the sum $\sum_{p \in L}(d_p + 1)$ is proportional to n^2. (See Exercise C-8.33.) Thus, algorithm height1 runs in $O(n^2)$ worst-case time.

We can compute the height of a tree more efficiently, in $O(n)$ worst-case time, by relying instead on the original recursive definition. To do this, we will parameterize a function based on a position within the tree, and calculate the height of the subtree rooted at that position. Algorithm height2, shown as nonpublic method _height2 in Code Fragment 8.5, computes the height of tree T in this way.

```
61   def _height2(self, p):                 # time is linear in size of subtree
62     """Return the height of the subtree rooted at Position p."""
63     if self.is_leaf(p):
64       return 0
65     else:
66       return 1 + max(self._height2(c) for c in self.children(p))
```

Code Fragment 8.5: Method _height2 for computing the height of a subtree rooted at a position p of a Tree.

It is important to understand why algorithm height2 is more efficient than height1. The algorithm is recursive, and it progresses in a top-down fashion. If the method is initially called on the root of T, it will eventually be called once for each position of T. This is because the root eventually invokes the recursion on each of its children, which in turn invokes the recursion on each of their children, and so on.

We can determine the running time of the height2 algorithm by summing, over all the positions, the amount of time spent on the nonrecursive part of each call. (Review Section 4.2 for analyses of recursive processes.) In our implementation, there is a constant amount of work per position, plus the overhead of computing the maximum over the iteration of children. Although we do not yet have a concrete implementation of children(p), we assume that such an iteration is generated in $O(c_p + 1)$ time, where c_p denotes the number of children of p. Algorithm height2 spends $O(c_p + 1)$ time at each position p to compute the maximum, and its overall running time is $O(\sum_p (c_p + 1)) = O(n + \sum_p c_p)$. In order to complete the analysis, we make use of the following property.

Proposition 8.5: *Let T be a tree with n positions, and let c_p denote the number of children of a position p of T. Then, summing over the positions of T, $\sum_p c_p = n - 1$.*

Justification: Each position of T, with the exception of the root, is a child of another position, and thus contributes one unit to the above sum. ■

By Proposition 8.5, the running time of algorithm height2, when called on the root of T, is $O(n)$, where n is the number of positions of T.

Revisiting the public interface for our Tree class, the ability to compute heights of subtrees is beneficial, but a user might expect to be able to compute the height of the entire tree without explicitly designating the tree root. We can wrap the non-public _height2 in our implementation with a public height method that provides a default interpretation when invoked on tree T with syntax T.height( ). Such an implementation is given in Code Fragment 8.6.

```
67   def height(self, p=None):
68     """Return the height of the subtree rooted at Position p.
69
70     If p is None, return the height of the entire tree.
71     """
72     if p is None:
73       p = self.root( )
74     return self._height2(p)          # start _height2 recursion
```

Code Fragment 8.6: Public method Tree.height that computes the height of the entire tree by default, or a subtree rooted at given position, if specified.